מקרא:
טקסט המסומן בצבע ירוק – הרחבות למי שמעוניין בכך.
החומר המתואר בחלקים אלו אינו מהווה חלק מהחומר המחייב לבחינה.

המבנה הפנימי של מערכת הקבצים: Inodes ו-Data Blocks

1. חלוקת השטח הפיזי: Block Groups וה- Superblock

החלוקה הפנימית של ה-NVM: ה-Superblock וה-Group Descriptors
כאשר אנו מעצבים מחיצה (לדוגמה, עם ext4), המערכת מחלקת את השטח הפיזי של ה-NVM לקבוצות בלוקים (Block Groups). כל קבוצה כזו היא יחידת ניהול עצמאית ובנויה מהשדות הבאים, הממוקמים בתחילתה:

1. Superblock (בלוק-על): זהו המידע החשוב ביותר. הוא מכיל את ה-Metadata הגלובלי של כל מערכת הקבצים: גודלה הכולל, מספר ה-Inodes הכולל, גודל בלוק הנתונים (למשל, 4096 בתים), ומצביע לבלוק הראשון של המחיצה. הרציונל: הליבה חייבת לדעת את "כללי המשחק" לפני שהיא קוראת כל דבר אחר. ה-Superblock משוכפל במספר Block Groups כדי להבטיח עמידות מפני קריסה.

ה-Superblock הוא באיזשהו מובן 'העיקר' של מערכת הקבצים. הוא מכיל את כל הנתונים הנדרשים לטיפול וטעינת המערכת.

2. Group Descriptors: מכיל את הכתובת של מבני הנתונים הפנימיים בתוך אותה Block Group (היכן מתחילה ה-Inode Table, היכן נמצא ה-Data Bitmap וכו').

3. Inode Bitmap: מפת הביטים לניהול ה-Inodes הפנויים.
4. Data Bitmap: מפת הביטים לניהול ה-Data Blocks הפנויים.
5. Inode Table: טבלת המטא-דאטה של הקבצים בפועל.
6. Data Blocks: הנתונים של הקבצים עצמם.

כאמור לעיל, פעולת ה-Formatting מחלקת את המחיצה (ה-Partition) לרצף של יחידות לוגיות הנקראות Block Groups (קבוצות בלוקים).
כל Block Group פועלת כיחידת ניהול קטנה ועצמאית, המאפשרת למנהל הקבצים:
1. לוקליות (Locality): שמירת ה-Metadata וה-Data של קובץ קרוב אחד לשני משפרת את ביצועי I/O (פחות תנועה של ראש הדיסק).
2. עמידות (Resilience): אם Block Group אחת נפגמת, שאר המערכת יכולה להמשיך לתפקד.
- תוכן קריטי: גודל המחיצה, מספר ה-Inodes הפנויים, גודל בלוק הנתונים, והיסטוריית השינויים האחרונה של המערכת.

© יצחק נודלר

- תפקיד קריטי (Consistency Check): ה-Kernel משתמש במידע זה כדי לבדוק את שלמות מערכת הקבצים (Consistency) לאחר קריסה.
- שכפול (Redundancy): ה-Superblock משוכפל במספר Block Groups שונות.

הרציונל: אם הקטע הפיזי בתחילת המחיצה (היכן שה-Superblock הראשון שוכן) נפגם, ניתן לטעון את ה-Superblock מאחד העותקים המשוכפלים ולשחזר את המערכת. זה חיוני לפרסיסטנטיות של ה-NVM.

כעת נתמקד בשני המרכיבים האחרונים, שהם ליבת ה-I/O של המערכת.

2. טבלת הקבצים (Inode Table): ה-Metadata

ה-Inode Table היא רשימה ממוספרת של רשומות [= structs] בגודל קבוע.
כל רשונה נקראית Inode.

רשומת ה- Inode הוא ה-Metadata של קובץ. של כל קובץ.
קובץ רגיל, ספרייה וכו'.
זוהי רשימה ממוספרת רציפה של אינדקסים, המתחילה במספר 1 או 2 (בהתאם למערכת הקבצים).

דרך חלופית לתאר את רשומת ה- Inode: כל Inode הוא ה"תעודת הזהות" וה"מפרט הטכני" של קובץ יחיד (קובץ רגיל, ספרייה, קובץ 'מיוחד' וכו').

היחס ל-FHS (Filesystem Hierarchy Standard):

המשתמש 'רואה' **שמות קבצים** (למשל, /etx/passwd).
מבחינת הליבה [= ה- kernel] – היא 'רואה' מספרי Inode.
התוכן של קובץ הספריה (למשל הספרייה ששמה /etc) שומר {'מכיל'} רק את המיפוי: שם קובץ+ Inode Number.
כל מידע אחר נלקח ישירות מה-Inode.
<u>עקרון הפרדת Metadata מנתונים:</u>
זוהי <u>אבן יסוד</u> בניהול הקבצים של לינוקס.
ה- Inode מאפשר לליבה לדעת את כל המידע הדרוש לה על הקובץ (הרשאות, מיקום, גודל) מבלי צורך לקרוא אפילו בית אחד של נתונים מהקובץ עצמו.

על תפקידה הקריטי של ה- Inode Table:

כאשר הליבה רוצה לגשת לקובץ, למשל לקרוא ממנו מידע או לכתוב אליו מידע וכו', היא חייבת לגשת תחילה ל- Inode שלו כדי לדעת: מי הבעלים שלו, מה הרשאות הגישה אליו, היכן הנתונים שלו מאוחסנים פיזית ב- NVM, וכו.'

© יצחק נודלר

מדוע טבלת ה- Inode שמורה ומנוהלת ב- NVMולא בזיכרון הראשי?

טבלת ה- Inodes מכילה מידע חיוני לשחזור המערכת וקריטי להבטחת שלמות הנתונים.
אם היא הייתה מאוחסנת ומנוהלת ב- MM, אזי התוכן שלה היה 'מתנדף' [= הולך לאיבוד !!!] עם כיבוי המחשב, וכתוצאה מכך מערכת הקבצים הייתה 'נהרסת' !!
השמירה והניבול שלה ב- NVM מבטיח שמידע זה יישמר באופן עקבי [= Persistent. Non volatile]

הערה:
השם בלינוקס Inode Table או: Inode List.

3.1.1 רשומת Inode: תיאור של השדות

כל Inode [= Index node] מייצג קובץ או ספרייה בודדת וכו'.
זהו מבנה נתונים שמנוהל ב- **NVM**, והואמייצג את ה-**Metadata** של כל קובץ.
המידע השמור ב-Inode הוא **קבוע בגודלו** (למשל 128 או 256 בתים ב-ext4).

אוסף השדות ברשומת ה- Inode איננו 'סתמי'.
כל שדה ברשומת ה- Inode קיים לצורך פונקציונליות ספציפית של מנהל הקבצים:

שדה ב-Inode	תיאור השדה והערך הנשמר	מדוע המידע כרחי למנהל הקבצים?
Inode Number	המספר הסידורי (האינדקס) של הרשומה ב-Inode Table.	**המזהה הקנוני (Canonical ID)**: כל קובץ מקושר ל-Inode Number שלו. הוא קבוע, מהיר לאיתור, ומאפשר גישה ישירה לכל שאר

© יצחק נודלר

שדה ב-Inode	תיאור השדה והערך הנשמר	מדוע המידע כרחי למנהל הקבצים?
		הנתונים.
File Type/Mode	קובע את סוג הקובץ (רגיל, ספרייה, קישור סמלי, צינור) ואת 9 סיביות **ההרשאות** (Read/Write/Execute ל-Owner, Group, Others).	חיוני להבטחת **אכיפת אבטחת ה-Multi-User** . ללא שדה זה, לא ניתן היה למנוע גישת משתמשים לא מורשים.
Mode/Permissions	**16 סיביות** המייצגות: 1) **סוג הקובץ** (רגיל, סוקט, צינור, קישור סמלי); 2) **9 סיביות ההרשאה** (Read/Write/Execute ל-Owner/Group/Others); 3 **סיביות מיוחדות** כגון **SUID/SGID** (הפעלת קובץ בהרשאות הבעלים/קבוצה).	**אכיפת אבטחת הליבה:** קריטי למנגנון `execve()` . כדי להחליט אם למשתמש יש הרשאת **Execute** להריץ תוכנה. סיביות SUID חיוניות להרצת תוכניות (כמו `passwd`) בהרשאות גבוהות.
UID / GID	**מספר המשתמש (User ID)** ומספר הקבוצה (Group ID) הבעלים של הקובץ.	קריטי לניהול הרשאות (סעיף 2.3). שדה זה קובע מי יכול לשנות את הקובץ.
Link Count	**מונה הקישורים <u>הקשיחים</u> (Hard Links)** המצביעים על Inode זה.	קריטי **למדיניות המחיקה**: הליבה יכולה למחוק קובץ (unlink(), ולשחרר את בלוקי הנתונים המוקצים לו ולהחזיר אותם למאגר ה-Free Bolcks, רק כאשר מונה זה מגיע לאפס. כל עוד יש קישור קשיח אחד לפחות, ה-Inode נשאר בחיים.

© יצחק נודלר

שדה ב-Inode	תיאור השדה והערך הנשמר	מדוע המידע כרחי למנהל הקבצים?
Size (in bytes)	גודל הקובץ בבתים.	מאפשר לליבה לדעת את גודל הקובץ בפועל ומתי להפסיק קריאה/כתיבה.
Timestamps (Atime, Mtime, Ctime) Access/Modify/Change Times	חותמות זמן (Timestamp) המציינות: זמן גישה אחרון (Atime), זמן שינוי נתונים אחרון (Mtime), וזמן שינוי Metadata אחרון (Ctime).	**Mtime** הוא המנגנון העיקרי לניהול אוטומציה ב- **Makefiles** **Ctime** חיוני לגיבויים שמחפשים שינויים במבנה.
Block Count	מספר הבלוקים **המוקצים בפועל** לקובץ זה ב-NVM.	עוזר למנהל הקבצים לדעת כמה שטח פיזי תופס הקובץ בפועל.
Pointers (Block Addresses)	מערך של 15 מצביעים מצביעים המכיל את הכתובות הפיזיות של **בלוקי הנתונים**. מערך זה ממומש כ: 12 מצביעים ישירים, מצביע עקיף יחיד, מצביע עקיף כפול, ומצביע עקיף משולש.	מקשר את המידע הלוגי (ה-Inode) למיקום הפיזי של הנתונים ב-NVM, ומאפשר גישה יעילה לקבצים גדולים. ממומש במנגנון Direct, Single, Double ו- Triple Indirect Pointers (יורחב בפרק על Inode Allocation).

© יצחק נודלר

Data Blocks 3.1.2 (בלוקי נתונים)

מהם Data Blocks ?
זהו הרוב המכריע של שטח ה-NVM.
זהו האזור ב- NVMשמכיל את התוכן בפועל [= 'האמיתי'] של הקבצים והספריות.
זהו רצף של יחידות אחסון בגודל קבוע.

שימוש ספציפי (ספריות):
בלוק הנתונים של הספרייה אינו מכיל נתונים 'רגילים', אלא **טבלה פנימית** של זוגות:
Inode Number, string

מבנה הנתונים לניהול בלוקי הנתונים הפנויים (Allocation Management)

מנהל הקבצים (הקוד ב-Kernel) חייב לדעת אילו בלוקים פנויים.
היינו, מנהל הקבצים צריך לשמור על רשימה מדויקת של בלוקים זמינים כדי למנוע דריסת נתונים (Overwrite) ולהקצות מקום לקבצים חדשים.

מפת הביטים (Data Bitmap):

מבנה הנתונים הנפוץ ביותר ב- ext4 לשמירת בלוקי הנתונים הפנויים הוא **Bitmap (מפת ביטים)** או **Free Block List**.

הוא ממוקם בתחילת כל Block Group.

מבנה:
זוהי רשימה ארוכה של ביטים.
כל ביט ברשימה מייצג בלוק נתונים אחד ב-NVM.
כל ביט כזה מציין בלוק פנוי (ניתן להקצאה), ו- מציין בלוק תפוס.
ביט = 1: הבלוק תפוס (מאוחסן בו קובץ).
ביט = 0: הבלוק פנוי וניתן להקצאה לקובץ חדש.

חשיבות: ה-Bitmap מאפשר ל- Kernel מאפשר ל-Kernel למצוא במהירות את הבלוק הפנוי הראשון (או הקרוב ביותר) לצורך הקצאה לקובץ חדש, ובכך מייעל את פעולות הכתיבה.

הקצאה מהירה:
השימוש במפת ביטים מאפשר ל-Kernel למצוא בלוק פנוי בפקודת Bitwise אחת.

אסטרטגיית הקצאה (Allocation Strategy): הקצאת בלוקים סמוכים

במקום לחפש את הבלוק הפנוי הראשון (First Fit), מנהל הקבצים של Ext4 משתמש ב- Bitmap כדי ליישם הקצאה ממוקדת (Extent-Based Allocation): הקצאת רצף של בלוקים רציפים יחד.

© יצחק נודלר

זה מבטיח שקבצים גדולים נשמרים בצורה רציפה (Contiguous) ככל האפשר, ומשפר את מהירות ה-I/O באופן דרמטי.

Data Blocks, ניהול הקצאה (Allocation Management) ופרגמנטציה

פרגמנטציה פנימית (Internal Fragmentation): אם גודל הקובץ הוא 2 בתים בלבד, וגודל בלוק הוא 4096 בתים, אזי ה- 4094 בתים הנותרים בבלוק הם בזבוז (Waste).

הקצאת בלוקים סמוכים

מנהל הקבצים מנסה להקצות בלוקים סמוכים (Contiguous) כדי להפחית את זמן הגישה הפיזי לדיסק.

3.1.3 Data Blocks וניהול Indirection (עקיפין)

Data Blocks מכילים את תוכן הקובץ בפועל. מנגנון ה-**Indirection** מאפשר ללינוקס לנהל קבצים גדולים באמצעות מבנה קבוע ב-Inode:

- **12 מצביעים ישירים:** מכסים קבצים קטנים במהירות.
- **מצביעים עקיפים (Single/Double/Triple Indirection):** מצביעים לבלוקים שאינם מכילים נתונים, אלא **רשימות של כתובות** לבלוקי נתונים נוספים. זה מגדיל משמעותית את טווח הכתובות הניתנות לגישה על ידי Inode אחד.

© יצחק נודלר